

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل بیستم: Web crawling and indexes

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

20.1 Overview

20.1.1 Features a crawler must provide

20.1.2 Features a crawler should provide

20.2 Crawling

20.2.1 Crawler architecture

20.2.2 DNS resolution

20.2.3 The URL frontier

20.3 Distributing indexes

20.4 Connectivity servers

20.5 Chapter 20 Summary

20، 20.1 و ویژگی‌های کرالر وب: از تئوری تا عمل

 گوگل: چگونه میلیاردها صفحه وب را فهرست می‌کنند؟

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول بهبود موتور جستجوی این شرکت هستید. هر روز میلیاردها صفحه وب توسط ربات‌های گوگل (Crawlers) بازدید و ایندکس می‌شوند. این یک فرآیند بسیار پیچیده

است، زیرا وب با سرعت فوق‌العاده در حال تغییر است.

چالش‌های اصلی که تیم شما با آن‌ها روبرو است:

- **دامنه‌های کرالر:** وبسایت‌هایی با محدودیت‌های مختلف (مانند فروشگاه‌های آنلاین با میلیون‌ها محصول).
- **تله‌های کرالر:** صفحاتی که به‌طور نامتناهی تولید می‌شوند و ربات‌ها را در حلقه‌های بی‌نهایت قرار می‌دهند.
- **سرورهای پرسرعت:** وبسایت‌هایی که بار ترافیکی بالایی دارند و به‌روزرسانی مداوم نیاز دارند.

🕷 ربات‌های گوگل: از تئوری تا عمل

ربات‌های گوگل (Googlebots) باید ویژگی‌های کلیدی که در کتاب ذکر شد را داشته باشند:

- **استحکام:** مقاومت در برابر **دام‌های کرالر** که به‌طور نامتناهی تولید می‌شوند.
- **ادب:** رعایت فایل robots.txt و محدود کردن نرخ درخواست به هر دامنه.
- **توزیع‌پذیری:** امکان اجرا روی چندین ماشین برای پوشش سریع‌تر وب.
- **مقیاس‌پذیری:** افزایش نرخ کرالینگ با اضافه کردن منابع.
- **کارایی:** استفاده بهینه از منابع سیستم (CPU، ذخیره، پهنای باند).
- **کیفیت:** اولویت‌دهی به صفحات **مفیدتر** (مثلاً با بر اساس اعتبار دامنه).
- **تازگی:** به‌روزرسانی مداوم صفحات بر اساس **نرخ تغییر** آن‌ها.
- **گسترش‌پذیری:** معماری ماژولار برای پشتیبانی از پروتکل‌ها یا فرمت‌های داده جدید.

این ویژگی‌ها به ربات‌های گوگل اجازه می‌دهند تا با چالش‌های بزرگ وب مقابله کنند و در عین حال از بار سرورها و شبکه به صورت بهینه استفاده کنند.

🚫 تله‌های کرالر: چالش‌های پیشرفته

یکی از بزرگترین چالش‌ها برای کرال‌های وب، **تله‌های کرالر** (Spider Traps) است. این صفحات به‌طور عمدی یا ناشیانه تولید می‌شوند تا ربات‌ها را به در حلقه‌های بی‌نهایت بیندازند.

انواع رایج تله‌های کرالر:

- **تله‌های تقویمی:** صفحاتی با لینک‌های بی‌پایان که کاربر را به صفحات بی‌نهایت هدایت می‌کنند.
- **تله‌های زمانی:** صفحاتی با رویدادهای زمانی بی‌پایان که کاربر را در وب نگه می‌دارند.
- **تله‌های نرم‌افزاری:** صفحاتی که نیاز به نصب نرم‌افزار برای نمایش محتوای کامل دارند.
- **تله‌های منطقی:** ساختارهای پیچیده با مسیرهای تکراری که ربات‌ها را در مسیرهای بی‌پایان به دام می‌اندازند.

ربات‌های گوگل با الگوریتم‌های پیشرفته خود را در برابر این تله‌ها مقاوم می‌کنند. برای مثال، اگر یک صفحه بیش از 1000 لینک خروجی داشته باشد، ربات ممکن است آن را به عنوان یک تله تقویمی شناسایی و از کرایلینگ کامل آن خودداری کند.

🏗️ معماری توزیع‌شده: برای کرایلینگ در مقیاس بزرگ

برای کرایلینگ وب در مقیاس بزرگ (مانند کل وب)، گوگل از معماری **توزیع‌شده** استفاده می‌کند. در این معماری:

- **URL Server**: مسئولیت تخصیص URLها به ربات‌های مختلف برای کرایلینگ موازی.
- **Crawler**: کامپیوترهایی که صفحات وب را بارگیری و تحلیل می‌کنند.
- **Links Database**: پایگاه داده‌ای که لینک‌های بین صفحات را ذخیره می‌کند.
- **Document Store**: ذخیره محتوای صفحات پس از تحلیل.
- **Indexer**: سیستمی‌هایی که ایندکس‌های جستجو را ایجاد و به‌روزرسانی می‌کنند.

این معماری به گوگل اجازه می‌دهد تا میلیاردها صفحه را به صورت موازی کرایلینگ کند و با سرعت بسیار بالا ایندکس‌ها را به‌روزرسانی کند. هر بخش می‌تواند روی ماشین‌های مختلف اجرا شود و به صورت مستقل به‌روزرسانی شود.

🌐 کاربردهای عملی: از موتورهای جستجو تا توصیه‌گر محتوا

کرایلینگ وب تنها مرحله اول از زنجیره بازیابی اطلاعات است. مرحله بعد، **ایندکس‌گذاری و جستجو** است که در آن کاربران می‌توانند به اطلاعات دسترسی پیدا کنند. در این مرحله نیز، رویکردهای یادگیری ماشین نقش کلیدی دارند.

کاربردهای عملی که از این رویکردها استفاده می‌کنند:

- **موتورهای جستجو**: برای رتبه‌بندی نتایج جستجو بر اساس ارتباط با پرس‌وجو کاربر.
- **سیستم‌های توصیه**: برای ارائه پیشنهادهاى شخصی‌سازی بر اساس تاریخچه و رفتار کاربر.
- **پلتفرم‌های تجارت الکترونیک**: برای ترکیب داده‌های مختلف (جستجو، خرید، بازدید محصول) برای ارائه پیشنهادهاى جامع.
- **شبکه‌های تحلیل داده**: برای درک الگوهای پیچیده در رفتار کاربران و بهبود الگوریتم‌ها.

این کاربردها نشان می‌دهند که چگونه کرایلینگ وب و ایندکس‌گذاری می‌توانند پایه‌ای برای سیستم‌های هوشمندتر باشند که تجربه کاربری را به شکل معنادارتری به کاربران ارائه دهند.

🕷️ 20.2 کرایلینگ: کاوشگران دنیای وب

🌐 سفر به دنیای وب: چگونه ربات‌های گوگل میلیاردها صفحه را کشف می‌کنند؟

تصور کنید در حال طراحی یک سیستم برای گوگل هستید که باید تمام صفحات وب را کشف و ایندکس کند. این فرآیند که به آن "کرایلینگ" می‌گویند، مانند سفر یک کاوشگر به دنیای ناشناخته وب است.

ربات‌های گوگل (Googlebots) این سفر را به این صورت انجام می‌دهند:

1. **نقطه شروع:** از چند URL معروف مانند `google.com`، `wikipedia.org` و `twitter.com`

شروع می‌کنند. این‌ها "مجموعه دانه" (seed set) نامیده می‌شوند.

2. **بازدید از صفحه:** ربات به این آدرس‌ها سر زده و محتوای صفحه را دریافت می‌کند.

3. **تحلیل محتوا:** متن صفحه برای ایندکس‌گذاری استخراج می‌شود و تمام لینک‌های داخل صفحه شناسایی می‌شوند.

4. **ادامه سفر:** لینک‌های جدید به صفی به نام "مرز" (URL Frontier) URL اضافه می‌شوند تا در آینده بازدید شوند.

این فرآیند به صورت بازگشتی ادامه می‌یابد و ربات‌های گوگل روزانه میلیاردها صفحه جدید را کشف و ایندکس می‌کنند.

🚀 چالش‌های عملی: از تئوری تا اجرا

در نگاه اول، کرایلینگ ممکن است ساده به نظر برسد، اما در عمل با چالش‌های بزرگی روبرو است. برای درک این چالش‌ها، به مثال زیر توجه کنید:

شرکت "دیجی‌کالا" می‌خواهد تمام محصولات خود را در گوگل ایندکس کند. با میلیون‌ها محصول و هزاران صفحه جدید هر روز، این کار به سادگی امکان‌پذیر نیست. ربات‌های گوگل باید:

- **مقیاس‌پذیر باشند:** بتوانند میلیاردها صفحه را در زمان معقولی پردازش کنند.
- **کارا باشند:** از منابع سیستم بهینه استفاده کنند.
- **مودب باشند:** به سرورها فشار نیاورند.
- **مقاوم باشند:** در برابر خطاها و تله‌های وب مقاومت کنند.
- **قابل گسترش باشند:** بتوانند با تغییرات وب سازگار شوند.

برای مثال، کralر Mercator که پایه بسیاری از کralرهای تجاری است، می‌تواند صدها صفحه در ثانیه را پردازش کند. این یعنی برای ایندکس کردن یک میلیارد صفحه، فقط چند هفته زمان نیاز است!

💡 ادب در دنیای دیجیتال: چرا ربات‌ها باید مودب باشند؟

شاید فکر کنید ربات‌ها می‌توانند هر طور که می‌خواهند به سرورها درخواست ارسال کنند، اما در واقعیت، این کار می‌تواند به سرورها آسیب بزند. برای درک این موضوع، به این مثال توجه کنید:

تصور کنید یک فروشگاه کوچک آنلاین دارید که روزانه ۱۰۰ بازدیدکننده دارد. ناگهان یک ربات به صورت همزمان ۱۰۰۰ درخواست به سرور شما ارسال می‌کند. این کار باعث می‌شود سرور شما از کار بیفتد و مشتریان واقعی نتوانند به فروشگاه شما دسترسی پیدا کنند.

برای جلوگیری از این مشکلات، کرالهای حرفه‌ای باید این قوانین را رعایت کنند:

- فقط یک اتصال همزمان به هر سرور.
- تأخیر چند ثانیه‌ای بین درخواست‌های متوالی به یک سرور.
- رعایت سیاست‌های فایل `robots.txt` که مشخص می‌کند کدام بخش‌های سایت نباید کرال شوند.

این قوانین به عنوان "ادب کرالر" (Crawler Politeness) شناخته می‌شوند و برای سلامت اکوسیستم وب ضروری هستند.

📡 کرالینگ مداوم: چگونه گوگل اطلاعات را به‌روز نگه می‌دارد؟

وب به سرعت در حال تغییر است و صفحات جدید دائماً ایجاد می‌شوند. برای اینکه موتورهای جستجو اطلاعات به‌روزی ارائه دهند، باید به صورت مداوم وب را کرال کنند.

برای مثال، سایت‌های خبری مانند `irna.ir` یا `isna.ir` هر چند دقیقه یک بار به‌روزرسانی می‌شوند. ربات‌های گوگل این سایت‌ها را بیشتر از صفحاتی که به ندرت تغییر می‌کنند (مانند یک مقاله علمی قدیمی) بازدید می‌کنند.

در **کرالینگ مداوم** (Continuous Crawling)، URL‌های بازدیدشده دوباره به مرز URL اضافه می‌شوند تا در آینده دوباره بازدید شوند. این فرآیند به گوگل اجازه می‌دهد تا همیشه جدیدترین اطلاعات را در اختیار کاربران قرار دهد.

In [1]:

```
import time
import random
from collections import deque, defaultdict
from urllib.parse import urlsplit

class WebCrawlerSimulator:
    """
    شبیه‌سازی یک کرالر وب ساده که
    - از یک مجموعه دانه شروع می‌کند
    - اضافه می‌کند URL لینک‌ها را استخراج و به مرز
    - برای هر میزبان، فقط یک درخواست همزمان انجام می‌دهد
    - بین درخواست‌ها به یک میزبان، تأخیر رعایت می‌شود
    """

    def __init__(self, seed_urls, politeness_delay=2):
        """
        مقداردهی اولیه کرالر.
        - seed_urls: لیست URL اولیه
        - politeness_delay: تأخیر حداقل بین درخواست‌ها به یک میزبان (ثانیه)
        """
        self.seed_urls = seed_urls
        self.politeness_delay = politeness_delay
        self.url_frontier = deque(seed_urls) # های بازدیدنشده URL صف
        self.fetched_urls = set() # های بازدیدشده URL مجموعه
        self.last_fetch_time = defaultdict(float) # آخرین زمان درخواست به هر میزبان
        self.host_queues = defaultdict(deque) # ها برای هر میزبان URL صف

    def extract_host(self, url):
        """URL استخراج نام میزبان از"""
        return urlsplit(url).netloc.lower()

    def simulate_fetch_page(self, url):
        """
        شبیه‌سازی دریافت صفحه و استخراج لینک‌ها.
        را دریافت و پارس کند HTML در واقعیت، این تابع باید
        """
        print(f"دریافت صفحه {url}")
        # شبیه‌سازی زمان پردازش
        time.sleep(0.1)

        # تولید لینک‌های تصادفی برای شبیه‌سازی
        host = self.extract_host(url)
        new_links = []
        for i in range(random.randint(0, 3)):
            new_links.append(f"http://{host}/page_{random.randint(1, 100)}")
        return new_links
```

```
def is_polite_to_fetch(self, host):
    """بررسی اینکه آیا می‌توان به این میزبان درخواست فرستاد یا خیر"""
    current_time = time.time()
    last_time = self.last_fetch_time[host]
    return (current_time - last_time) >= self.politeness_delay

def crawl(self, max_pages=10):
    """
    اجرای فرایند کرالینگ تا حداکثر تعداد صفحات مشخص‌شده
    """
    print(f"اولیه URL {len(self.seed_urls)} شروع کرالینگ از")
    pages_fetched = 0

    while self.url_frontier and pages_fetched < max_pages:
        # ها در صف‌های میزبان URL توزیع
        while self.url_frontier:
            url = self.url_frontier.popleft()
            if url in self.fetched_urls:
                continue
            host = self.extract_host(url)
            self.host_queues[host].append(url)
            self.fetched_urls.add(url) # جلوگیری از بازدید مجدد

        # پردازش هر میزبان که شرایط ادب را دارد
        hosts_ready = [host for host in self.host_queues if self.is_polite_to_fetch(host)]

        if not hosts_ready:
            # اگر هیچ میزبانی آماده نیست، کمی صبر کن
            time.sleep(0.5)
            continue

        # انتخاب یک میزبان آماده
        host = random.choice(hosts_ready)
        if not self.host_queues[host]:
            continue

        url = self.host_queues[host].popleft()
        self.last_fetch_time[host] = time.time()

        # شبیه‌سازی دریافت صفحه و استخراج لینک‌ها
        new_links = self.simulate_fetch_page(url)
        pages_fetched += 1

        # اضافه کردن لینک‌های جدید به مرز
        for link in new_links:
            if link not in self.fetched_urls:
                self.url_frontier.append(link)

        print(f"لینک‌های جدید: {len(new_links)} | صفحات بازدیدشده: {pages_fetched}/{max_pages}")
        print("-" * 50)

    print(f"کرالینگ به پایان رسید. تعداد کل صفحات بازدیدشده: {pages_fetched}")

# مثال اجرا
if __name__ == "__main__":
    seed_urls = [
        "http://example.com/home",
        "http://sample.org/index",
        "http://test.net/main"
    ]

    crawler = WebCrawlerSimulator(seed_urls, politeness_delay=1)
    crawler.crawl(max_pages=8)

    print("\nنکته:")
    print("این شبیه‌سازی، مکانیزم اصلی کرالینگ را نشان می‌دهد:")
    print("URL (URL Frontier) مدیریت مرز -")
    print("رعایت ادب میزبان (یک اتصال، تأخیر بین درخواست‌ها) -")
    print("های یکسان URL جلوگیری از بازدید مجدد از -")
    print("و. پایگاه داده است HTML، HTTP در پیاده‌سازی واقعی، نیاز به پردازش")
```

...اولیه URL شروع کرالینگ از 3
دریافت صفحه: http://test.net/main
لینک‌های جدید: 2 | صفحات بازدیدشده: 1/8

دریافت صفحه: http://example.com/home
لینک‌های جدید: 1 | صفحات بازدیدشده: 2/8

دریافت صفحه: http://sample.org/index
لینک‌های جدید: 0 | صفحات بازدیدشده: 3/8

کرالینگ به پایان رسید. تعداد کل صفحات بازدیدشده: 3

نکته:

این شبیه‌سازی، مکانیزم اصلی کرالینگ را نشان می‌دهد:

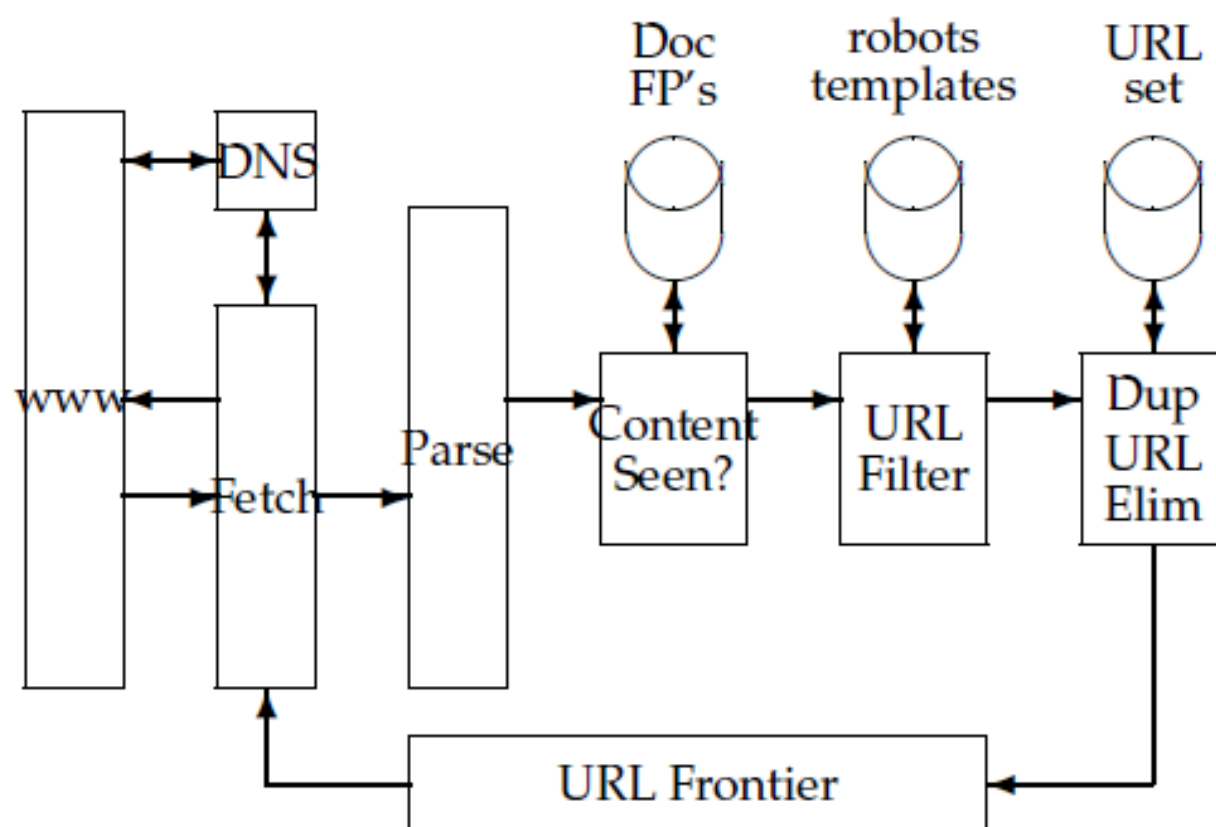
- URL (URL Frontier) مدیریت مرز
- رعایت ادب میزبان (یک اتصال، تأخیر بین درخواست‌ها)
- های یکسان URL جلوگیری از بازدید مجدد از
- و. پایگاه داده است HTML، HTTP در پیاده‌سازی واقعی، نیاز به پردازش

20.2.1 معماری کرالر وب: آناتومی یک کاوشگر دیجیتال

🔍 پشت صحنه گوگل: چگونه میلیاردها صفحه در روز ایندکس می‌شوند؟

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول طراحی کرالری هستید که باید هر روز میلیاردها صفحه وب را ایندکس کند. این کرالر مانند یک ارتش کوچک از ربات‌های هوشمند عمل می‌کند که هر کدام وظیفه خاصی دارند.

معماری این کرالر از چندین ماژول اصلی تشکیل شده است که با هم هماهنگ کار می‌کنند:



► Figure 20.1 The basic crawler architecture.

شکل 20.1: معماری پایه خزنده وب .

📋 مرز URL: صف انتظار دیجیتالی

مرز (URL Frontier) مانند یک صف انتظار بی‌پایان است که تمام URL‌هایی که باید بازدید شوند را نگهداری می‌کند. در کرالینگ مداوم، حتی URL‌هایی که قبلاً بازدید شده‌اند نیز به این صف بازمی‌گردند تا در آینده دوباره بررسی شوند.

برای مثال، وقتی گوگل سایت دیجی‌کالا را کرال می‌کند، تمام صفحات محصولات در این صف قرار می‌گیرند. با توجه به اینکه قیمت‌ها و موجودی محصولات به طور مداوم تغییر می‌کنند، این URL‌ها به طور منظم به صف بازگردانده می‌شوند تا اطلاعات به‌روز باقی بمانند.

🌐 ماژول DNS: دفترچه تلفن اینترنت

قبل از اینکه کرالر بتواند صفحه‌ای را دریافت کند، باید آدرس IP سرور را پیدا کند. این کار توسط ماژول DNS انجام می‌شود که مانند دفترچه تلفن اینترنت عمل می‌کند.

برای مثال، وقتی کرالر می‌خواهد صفحه `https://www.bing.com/search?q=crawling` را دریافت کند، ابتدا باید آدرس IP سرور `bing.com` را پیدا کند. این فرآیند ممکن است چند میلی‌ثانیه طول بکشد، اما برای کرالرهایی که میلیاردها درخواست ارسال می‌کنند، این زمان اضافی می‌تواند تأثیر زیادی بر کارایی داشته باشد.

📡 مازول Fetch: پیک اینترنت

ماژول Fetch مانند یک پیک دیجیتال است که به سرورها مراجعه کرده و محتوای صفحات وب را تحویل می‌گیرد. این مازول از پروتکل HTTP برای برقراری ارتباط با سرورها استفاده می‌کند.

برای مثال، وقتی کرالر گوگل می‌خواهد صفحه اصلی سایت ورزش سه را دریافت کند، یک درخواست HTTP به سرور ورزش سه ارسال می‌کند و سرور محتوای HTML صفحه را به کرالر برمی‌گرداند.

📄 مازول Parse: خواننده هوشمند

ماژول Parse مانند یک خواننده هوشمند عمل می‌کند که محتوای HTML را تحلیل کرده و اطلاعات مهم را استخراج می‌کند. این مازول متن اصلی صفحه، لینک‌ها و سایر اطلاعات مفید را جدا می‌کند.

برای مثال، وقتی کرالر صفحه اصلی سایت تپسل (`tapsell.ir`) را تحلیل می‌کند، این مازول تمام لینک‌های موجود در صفحه را استخراج کرده و برای بررسی‌های بعدی به مرز URL اضافه می‌کند. همچنین، متن اصلی صفحه برای ایندکس‌گذاری به سیستم جستجو ارسال می‌شود.

🚫 حذف محتوای تکراری: جلوگیری از اتلاف منابع

یکی از چالش‌های بزرگ کرالرها، جلوگیری از پردازش محتوای تکراری است. این کار با استفاده از اثر انگشت (Fingerprint) یا Shingling انجام می‌شود.

برای مثال، ممکن است یک مقاله خبری در چندین سایت خبری مختلف منتشر شود. کرالر باید بتواند تشخیص دهد که این محتوا تکراری است و از پردازش مجدد آن خودداری کند. این کار با محاسبه اثر انگشت محتوا و مقایسه آن با اثر انگشت‌های موجود انجام می‌شود.

🤖 احترام به حریم خصوصی: پروتکل Robots.txt

وب‌مسترها می‌توانند با استفاده از فایل `robots.txt` مشخص کنند که کدام بخش‌های سایتشان نباید توسط کرالرها بازدید شود. این فایل مانند یک تابلو "ورود ممنوع" برای ربات‌ها عمل می‌کند.

برای مثال، فایل `robots.txt` سایت آمازون ممکن است مشخص کند که کرالرها نباید صفحات پرداخت و اطلاعات شخصی کاربران را بازدید کنند. این فایل باید مستقیماً قبل از دریافت صفحه بررسی شود تا از نقض قوانین سایت جلوگیری شود.

در اینجا یک نمونه از فایل `robots.txt` سایت توییتر آمده است:

```
* :User-agent
Disallow: /search
Disallow: /sms
Disallow: /sessions
Disallow: /settings

User-agent: Googlebot
Allow: /hashtags
/Allow: /i
/Allow: /intent
```

🌐 کرایینگ توزیع‌شده: کار تیمی در مقیاس جهانی

برای کرایینگ وب در مقیاس بزرگ، شرکت‌هایی مانند گوگل و مایکروسافت از معماری توزیع‌شده استفاده می‌کنند. در این معماری، کرالرها در سراسر جهان توزیع شده‌اند و هر کرالر مسئولیت کرایینگ مجموعه‌ای از سایت‌ها را بر عهده دارد.

برای مثال، گوگل ممکن است کرالرهایی در اروپا داشته باشد که کرایینگ سایت‌های اروپایی، و کرالرهایی در آسیا که سایت‌های آسیایی را کرال می‌کنند. این توزیع جغرافیایی به کاهش تأخیر و افزایش کارایی کمک می‌کند.

در این معماری، یک "تقسیم‌کننده میزبان" (Host Splitter) مشخص می‌کند که هر URL به کدام کرالر ارسال شود. این تقسیم‌بندی معمولاً بر اساس هش نام دامنه انجام می‌شود تا توزیع بار به صورت یکنواخت انجام شود.

⚡ چالش‌های عملی: از تئوری تا پیاده‌سازی

پیاده‌سازی یک کرالر در مقیاس بزرگ با چالش‌های زیادی همراه است:

- **مدیریت حافظه:** مرز URL ممکن است حاوی میلیاردها آدرس باشد که نیاز به مدیریت حافظه هوشمند دارد.
- **تحمل خطا:** اگر یک کرالر از کار بیفتد، باید مکانیزمی برای ادامه کار از آخرین نقطه وجود داشته باشد.
- **توزیع بار:** باید اطمینان حاصل شود که هیچ سروری تحت فشار بیش از حد قرار نگیرد.
- **به‌روزرسانی مداوم:** وب دائماً در حال تغییر است و کرالر باید بتواند این تغییرات را پیگیری کند.

برای مقابله با این چالش‌ها، شرکت‌های بزرگ از سیستم‌های پیچیده‌ای استفاده می‌کنند که شامل مکانیزم‌های ذخیره‌سازی توزیع‌شده، الگوریتم‌های هوشمند برای اولویت‌بندی URL‌ها و سیستم‌های نظارتی پیشرفته است.

20.2.2 تحلیل DNS: دفترچه تلفن اینترنت 🌐

📱 از آمازون تا گوگل: چرا DNS برای کرالرها حیاتی است؟

تصور کنید می‌خواهید به دوستان پیام دهید، اما فقط شماره تلفن او را دارید، نه نامش. حالا برعکس: فقط نام دوستان را می‌دانید، اما شماره تلفنش را نه. این دقیقاً همان چیزی است که در اینترنت اتفاق می‌افتد.

وقتی در مرورگر خود `www.amazon.com` را تایپ می‌کنید، کامپیوتر شما نمی‌داند که این سایت در کجای اینترنت قرار دارد. باید نام دامنه به آدرس IP عددی (مثل `174.129.25.17`) تبدیل شود. این فرآیند را **تحلیل DNS** (DNS Resolution) می‌نامیم.

🕒 گلوگاه سرعت: چرا DNS کرالرها را کند می‌کند؟

برای یک کاربر عادی، تحلیل DNS فقط چند میلی‌ثانیه طول می‌کشد و اصلاً قابل توجه نیست. اما برای یک کرالر که باید در ثانیه صدها صفحه را دریافت کند، این فرآیند می‌تواند یک فاجعه باشد!

چرا؟

- تحلیل DNS ممکن است **چند ثانیه** طول بکشد
- برای دامنه‌های ناشناخته یا مشکوک، این زمان حتی بیشتر هم می‌شود
- اگر کرالر بخواهد ۱۰۰۰ صفحه در ثانیه دریافت کند، هر تأخیر چند ثانیه‌ای یک مانع بزرگ است

🏠 راه‌حل اولیه: کش کردن نتایج DNS

اولین راه‌حلی که به ذهن می‌رسد، **کش کردن** نتایج DNS است. مانند اینکه شماره تلفن دوستان را در دفترچه تلفن خود یادداشت کنید تا دیگر نیازی به جستجو نداشته باشید.

اما این راه‌حل محدودیت‌هایی دارد:

- به دلیل **محدودیت‌های ادب کرالینگ** (فاصله بین درخواست‌ها به یک سرور)، نرخ استفاده مجدد از کش کاهش می‌یابد
- آدرس‌های IP ممکن است تغییر کنند، بنابراین کش‌ها باید به‌طور منظم به‌روز شوند
- برای کرال‌های بزرگ، حتی کش هم کافی نیست

🚫 مشکل عمیق‌تر: رفتار همگام کتابخانه‌های DNS

مشکل اصلی اینجاست: کتابخانه‌های استاندارد DNS **همگام** (Synchronous) عمل می‌کنند. یعنی وقتی یک کرالر درخواست DNS ارسال می‌کند، تمام فعالیت‌های دیگر متوقف می‌شوند تا پاسخ دریافت شود.

این مانند این است که در یک رستوران، گارسون فقط بتواند یک سفارش را بگیرد و تا زمانی که غذا برای آن میز آماده نشده، از میزهای دیگر سفارش نگیرد!

🏢 راه‌حل گوگل: DNS غیرهمگام سفارشی

شرکت‌هایی مانند گوگل و آمازون که کرال‌های بسیار بزرگی دارند، این مشکل را با پیاده‌سازی **resolver DNS سفارشی و غیرهمگام** حل کرده‌اند. این سیستم چگونه کار می‌کند؟

1. هر رشته کرالر، یک پیام DNS ارسال کرده و به کار خود ادامه می‌دهد (منتظر نمی‌ماند)

2. یک رشته اختصاصی DNS به طور مداوم پاسخ‌ها را دریافت کرده و به رشته‌های مربوطه تحویل می‌دهد

3. اگر پاسخی در زمان مشخصی (مثلاً ۱ ثانیه) دریافت نشد، درخواست مجدداً ارسال می‌شود

4. این تلاش‌ها معمولاً تا ۵ بار تکرار می‌شوند و زمان انتظار به صورت نمایی افزایش می‌یابد (از ۱ ثانیه تا ۹۰ ثانیه)

این طراحی به کرالر اجازه می‌دهد تا به جای انتظار برای پاسخ‌های DNS، به کارهای دیگر ادامه دهد و در نتیجه سرعت کرالینگ به شدت افزایش می‌یابد.

🎯 نتیجه‌گیری: چرا این موضوع مهم است؟

بدون راه‌حل‌های هوشمند برای تحلیل DNS، کرالرهای مدرن نمی‌توانستند با سرعت مورد نیاز عمل کنند. این به‌ویژه برای شرکت‌هایی مانند گوگل که باید میلیاردها صفحه را در روز ایندکس کنند، حیاتی است.

در واقع، بهینه‌سازی DNS یکی از رازهای موفقیت کرالرهای بزرگ است که به آنها اجازه می‌دهد وب را با سرعت فوق‌العاده‌ای پوشش دهند.

20.2.3 URL: رهبر ترافیک اینترنت 🚦

🌐 چالش گوگل: چگونه میلیاردها صفحه را منظم کرال کنیم؟

تصور کنید رهبر تیم کرالینگ گوگل هستید. هر ثانیه، هزاران صفحه جدید به وب اضافه می‌شود و شما باید تصمیم بگیرید کدام صفحه را اول بازدید کنید. این مثل مدیریت ترافیک یک کلان شهر است که در آن همه جاده‌ها یک‌باره شلوغ شده‌اند.

شما با دو چالش بزرگ روبرو هستید:

1. **اولویت‌دهی هوشمند:** صفحاتی که مهم هستند و به سرعت تغییر می‌کنند (مانند خبرهای فوری در bbc.com یا قیمت‌های روز در amazon.com) باید سریع‌تر از صفحات کم‌اهمیت‌تر کرال شوند.
2. **ادب در ترافیک:** شما نباید با ارسال همزمان صدها درخواست به یک سرور (مثلاً apple.com در زمان عرضه یک محصول جدید)، آن را از کار بیندازید.

🏗️ راه‌حل: معماری دو لایه مرز URL

برای حل این چالش، کرالرهای مدرن از یک سیستم صف هوشمند به نام **مرز URL** (URL Frontier) استفاده می‌کنند. این سیستم مانند یک مرکز کنترل ترافیک پیشرفته عمل می‌کند که از دو بخش اصلی تشکیل شده است:

- **صف‌های جلو (Front Queues):** این صف‌ها مانند خطوط ویژه وسایل نقلیه اضطراری عمل می‌کنند. هر کدام از این صف‌ها برای سطح اولویت خاصی طراحی شده‌اند. برای مثال، URL‌های مربوط به خبرهای فوری در بالاترین صف قرار می‌گیرند، در حالی که URL‌های وبلاگ‌های شخصی در صف‌های با اولویت پایین‌تر قرار می‌گیرند.
- **صف‌های عقب (Back Queues):** این صف‌ها مانند چراغ‌های راهنمایی برای هر خیابان (میزبان) خاص عمل می‌کنند. هر صف عقب فقط شامل URL‌های یک میزبان خاص است. این تضمین می‌کند که در هر لحظه فقط یک درخواست به یک سرور خاص ارسال شود.

🔗 هماهنگ‌کننده اصلی: هیپ زمان‌بندی

بین این دو لایه، یک هیپ (Heap) قرار دارد که مانند یک ساعت هوشمند عمل می‌کند. این هیپ برای هر میزبان، زودترین زمانی که می‌توان دوباره به آن درخواست ارسال کرد را ذخیره می‌کند.

برای مثال، اگر آخرین بار در ساعت ۱۰:۰۰:۰۰ به سرور `nytimes.com` درخواست ارسال شده و این درخواست ۲ ثانیه طول کشیده باشد، هیپ ممکن است زمان مجاز بعدی را ساعت ۱۰:۰۰:۲۰ (یعنی ۱۰ برابر زمان درخواست قبلی) تنظیم کند.

📍 سفر یک URL از ورود تا خروج

بیایید سفر یک URL خبری از `reuters.com` را دنبال کنیم:

1. **ورود و اولویت‌دهی:** URL خبری به سیستم وارد شده و به دلیل اهمیت بالای خبر، به بالاترین صف جلو اضافه می‌شود.
2. **انتقال به صف عقب:** وقتی یک صف عقب خالی می‌شود، یک "کنترل‌کننده هوشمند" URL را از صف‌های جلو (با تمایل به صف‌های با اولویت بالاتر) برداشته و آن را به صف عقب مربوط به `reuters.com` اضافه می‌کند.
3. **انتظار برای زمان مجاز:** کرالر به هیپ نگاه می‌کند و می‌بیند که آیا زمان مجاز برای ارسال درخواست به `reuters.com` فرا رسیده است یا خیر.
4. **کرایلینگ:** پس از رسیدن به زمان مجاز، کرالر URL را از صف عقب برداشته و صفحه را دریافت می‌کند.
5. **به‌روزرسانی زمان:** پس از اتمام کرایلینگ، زمان مجاز بعدی برای `reuters.com` در هیپ به‌روزرسانی می‌شود.

📁 مدیریت حجم عظیم: وقتی حافظه RAM کافی نیست

در کرایلینگ در مقیاس وب، مرز URL ممکن است به میلیاردها آدرس برسد که بسیار بزرگ‌تر از حافظه RAM یک سرور است. راه‌حل این است که بخش اعظم مرز URL روی **دیسک** ذخیره شود.

این مانند یک پارکینگ خیلی بزرگ است که اکثر خودروها در آن منتظر می‌مانند و فقط تعداد محدودی در مرکز کنترل ترافیک (حافظه RAM) برای پردازش فوری قرار دارند. این طراحی به کralرها اجازه می‌دهد تا کل وب را مدیریت کنند، حتی با منابع محاسباتی محدود.

```
In [8]: import heapq
import time
import random
from collections import deque, defaultdict

class URLFrontierSimulator:
    """
    با دو لایه URL شبیه‌سازی مرز:
    - برای اولویت‌دهی بر اساس کیفیت و نرخ تغییر (Front Queues) صف‌های جلو -
    - برای رعایت ادب (یک میزبان در هر صف) (Back Queues) صف‌های عقب -
    - هیپ زمان‌بندی: برای مدیریت زمان مجاز درخواست به هر میزبان -
    """

    def __init__(self, num_front_queues=3, num_back_queues=10):
        # لایه اولویت‌دهی
        self.front_queues = [deque() for _ in range(num_front_queues)] # F صف جلو
        self.num_front = num_front_queues

        # لایه ادب
        self.back_queues = [deque() for _ in range(num_back_queues)] # B صف عقب
        self.host_to_back_queue = {} # نگاشت میزبان -> شماره صف عقب
        self.next_back_queue = 0 # برای تخصیص چرخشی صف عقب جدید

        # هیپ زمان‌بندی: (زمان_مجاز, شماره_صف_عقب)
        self.time_heap = []

        # سایر تنظیمات
        self.host_last_fetch = {} # آخرین زمان درخواست به هر میزبان
        self.host_fetch_duration = defaultdict(lambda: 1.0) # مدت‌زمان تخمینی Fetch

    def extract_host(self, url):
        """(سادشده) URL استخراج نام میزبان از"""
        if "://" in url:
            url = url.split("://", 1)[1]
        return url.split("/")[0].lower()

    def assign_priority(self, url):
        """
        (بالاترین = self.num_front-1، پایین‌ترین = ۰) URL تخصیص اولویت به
        در واقعیت، این تابع بر اساس تاریخچه تغییر و کیفیت صفحه تصمیم می‌گیرد.
        """
        host = self.extract_host(url)
        # شبیه‌سازی: دامنه‌های خبری اولویت بالاتری دارند
        if any(keyword in host for keyword in ["news", "bbc", "cnn"]):
            return self.num_front - 1
        # صفحات معمولی اولویت تصادفی دریافت می‌کنند
        return random.randint(0, self.num_front - 2)

    def add_url(self, url):
        """جدید به مرز URL افزودن"""
        priority = self.assign_priority(url)
        self.front_queues[priority].append(url)
        print(f" {priority} {url} به صف جلو URL افزودن")

    def _get_or_create_back_queue(self, host):
        """دریافت یا ایجاد صف عقب برای یک میزبان"""
        if host in self.host_to_back_queue:
            return self.host_to_back_queue[host]

        # تخصیص یک صف عقب جدید
        back_queue_id = self.next_back_queue % len(self.back_queues)
        self.next_back_queue += 1
        self.host_to_back_queue[host] = back_queue_id

        # اولین زمان مجاز برای این میزبان
        current_time = time.time()
        heapq.heappush(self.time_heap, (current_time, back_queue_id))

        return back_queue_id

    def _refill_back_queue(self, back_queue_id):
        """پر کردن یک صف عقب خالی از صف‌های جلو (با اولویت بالاتر)"""
        # جستجوی صف‌های جلو از اولویت بالا به پایین
        for priority in range(self.num_front - 1, -1, -1):
            if self.front_queues[priority]:
                url = self.front_queues[priority].popleft()
                host = self.extract_host(url)
                target_back_id = self._get_or_create_back_queue(host)

                # را به صف عقب صحیح خودش اضافه کن
                self.back_queues[target_back_id].append(url)

        # اگر صفی که پر کردیم همان صف درخواستی بود، کار تمام است
```

```
        if target_back_id == back_queue_id:
            return True
        # در غیر این صورت، به جستجو برای پر کردن صف درخواستی ادامه بده
    return False

def get_next_url(self):
    """بعدی برای کرالینگ (با رعایت ادب و اولویت) URL دریافت"""
    # اگر هیچ زمان‌بندی وجود ندارد، سعی کن یک صف را برای شروع پر کنی
    if not self.time_heap:
        # تمام صف‌های عقب را با بالاترین اولویت ممکن پر کن
        for i in range(len(self.back_queues)):
            self._refill_back_queue(i)

        # ای وجود ندارد URL اگر هنوز هیپ خالی است، یعنی هیچ
        if not self.time_heap:
            return None

    # دریافت زمان مجاز بعدی
    earliest_time, back_queue_id = heapq.heappop(self.time_heap)
    current_time = time.time()

    # اگر هنوز زمان مجاز نرسیده، منتظر بمان
    if current_time < earliest_time:
        time.sleep(earliest_time - current_time)
        current_time = earliest_time

    # اگر صف عقب خالی است، سعی کن آن را پر کنی
    if not self.back_queues[back_queue_id]:
        if not self._refill_back_queue(back_queue_id):
            # ی باقی نمانده URL اگر نتوانستیم آن را پر کنیم، یعنی هیچ
            return None

    # اگر بعد از پر کردن، هنوز هم صف خالی بود، به سراغ زمان بعدی برو
    if not self.back_queues[back_queue_id]:
        return self.get_next_url()

    # از صف عقب URL دریافت
    url = self.back_queues[back_queue_id].popleft()
    host = self.extract_host(url)

    # به‌روزرسانی زمان‌ها
    self.host_last_fetch[host] = current_time
    fetch_duration = self.host_fetch_duration[host]
    next_allowed_time = current_time + 10 * fetch_duration # سیاست Mercator

    # زمان مجاز بعدی را به هیپ اضافه کن
    heapq.heappush(self.time_heap, (next_allowed_time, back_queue_id))

    return url

# مثال استفاده
if __name__ == "__main__":
    # با 3 صف ادب URL ایجاد مرز
    frontier = URLFrontierSimulator(num_front_queues=3, num_back_queues=5)

    # های نمونه URL افزودن
    sample_urls = [
        "http://news-example.com/latest",
        "http://blog-site.org/post1",
        "http://cnn-breaking.news/update",
        "http://regular-site.com/page",
        "http://bbc-world.news/today",
        "http://another-blog.net/article"
    ]

    print("ها به مرز URL افزودن")
    for url in sample_urls:
        frontier.add_url(url)

    print("\nشبیه‌سازی کرالینگ")

    # حالا حلقه اصلی کرالینگ را اجرا می‌کنیم
    for i in range(6):
        url = frontier.get_next_url()
        if url:
            print(f"کرالینگ: {url}")
            # شبیه‌سازی زمان پردازش
            time.sleep(0.1)
        else:
            print("ی برای کرالینگ باقی نمانده URL هیچ")
            break

    print("\nنکته")
    print("را نشان می‌دهد URL این شبیه‌سازی مکانیزم اصلی مرز")
    print("اولیت‌دهی بر اساس میزبان (دامنه‌های خبری اولویت بالاتر) -")
    print("رعایت ادب با صف‌های میزبان‌محور -")
    print("مدیریت زمان‌بندی با هیپ -")
    print("در پیاده‌سازی واقعی، اولویت‌ها بر اساس تاریخچه تغییر صفحه محاسبه می‌شوند")
```

ها به مرزURL افزودن

- به صف جلو (اولویت 2) URL افزودن http://news-example.com/latest
- به صف جلو (اولویت 1) URL افزودن http://blog-site.org/post1
- به صف جلو (اولویت 2) URL افزودن http://cnn-breaking.news/update
- به صف جلو (اولویت 0) URL افزودن http://regular-site.com/page
- به صف جلو (اولویت 2) URL افزودن http://bbc-world.news/today
- به صف جلو (اولویت 0) URL افزودن http://another-blog.net/article

شبیهسازی کرالینگ:

- کرالینگ: http://news-example.com/latest
- کرالینگ: http://cnn-breaking.news/update
- کرالینگ: http://bbc-world.news/today
- کرالینگ: http://blog-site.org/post1
- کرالینگ: http://regular-site.com/page
- کرالینگ: http://another-blog.net/article

نکته:

را نشان می‌دهد URL این شبیهسازی مکانیزم اصلی مرز اولویت‌دهی بر اساس میزبان (دامنه‌های خبری اولویت بالاتر) - رعایت ادب با صف‌های میزبان‌محور - مدیریت زمان‌بندی با هیپ - در پیاده‌سازی واقعی، اولویت‌ها بر اساس تاریخچه تغییر صفحه محاسبه می‌شوند

چند کرالر واقعی

In [12]:

```
import time
import random
import re
from collections import deque, defaultdict
from urllib.parse import urljoin, urlparse

# --- توابع کمکی برای شبیه‌سازی ---

def simulate_dns_lookup(host):
    """با تأخیر تصادفی DNS شبیه‌سازی حل"""
    print(f" [DNS] در حال پیدا کردن آدرس {host}...")
    time.sleep(random.uniform(0.1, 0.5)) # شبیه‌سازی تأخیر شبکه
    return f"192.168.1.{random.randint(1, 254)}"

def simulate_http_fetch(url):
    """شبیه‌سازی دریافت صفحه با تأخیر تصادفی"""
    print(f" [HTTP] در حال دریافت صفحه از {url}...")
    time.sleep(random.uniform(0.5, 1.5)) # شبیه‌سازی تأخیر دریافت
    # ساختگی برای شبیه‌سازی برمی‌گردانیم HTML یک محتوای
    return f"""
<html>
<head><title>Page at {url}</title></head>
<body>
  <h1>Welcome to {urlparse(url).netloc}</h1>
  <p>This is a sample page.</p>
  <a href="{urljoin(url, '/page1.html')}">Link 1</a><br>
  <a href="{urljoin(url, '/page2.html')}">Link 2</a><br>
  <a href="http://external-site.com/about">External Link</a>
</body>
</html>
"""

def extract_links(html_content, base_url):
    """(ساده‌شده) HTML استخراج لینک‌ها از محتوای"""
    # پیدا کردن تمام لینک‌ها در تگ <a href="...">
    links = re.findall(r'<a\s+(?:[^\s]*)href="([^\"]*)"', html_content, re.IGNORECASE)
    # تبدیل لینک‌های نسبی به مطلق
    absolute_links = [urljoin(base_url, link) for link in links]
    return list(set(absolute_links)) # حذف لینک‌های تکراری

def check_robots_txt(url, robots_cache):
    """URL برای یک robots.txt بررسی فایل"""
    host = urlparse(url).netloc
    if host in robots_cache:
        print(f" [Robots] استفاده از کش robots.txt برای {host}")
        return robots_cache[host]

    print(f" [Robots] در حال دریافت robots.txt برای {host}...")
    time.sleep(0.2) # شبیه‌سازی دریافت فایل
    # ساده که همه چیزها مجاز هستند robots.txt شبیه‌سازی یک فایل
    robots_cache[host] = {'allowed': True}
    return robots_cache[host]

# --- کلاس اصلی کرالر ---

class WebCrawler:
    """
    ،ادب URL یک کرالر وب ساده که مفاهیم اصلی مانند مرز
    را پیاده‌سازی می‌کند robots.txt حذف تکراری و احترام به

    """

    def __init__(self, seed_urls, max_pages=20, politeness_delay=3):
        """
        مقداردهی اولیه کرالر
        seed_urls: لیست URL شروع
        max_pages: حداکثر تعداد صفحات برای دریافت
        politeness_delay: تأخیر به ثانیه بین درخواست‌ها به یک میزبان
        """

        self.max_pages = max_pages
        self.politeness_delay = politeness_delay
```

```
# ها که باید بازدید شوندURL صفی از :URL مرز
self.url_frontier = deque(seed_urls)

# هایی که قبلاً دیده‌ایم (برای جلوگیری از تکرار)URL مجموعه‌ای از
self.visited_urls = set()

# هر میزبان robots.txt کش برای فایل‌های
self.robots_cache = {}

# صف برای هر میزبان برای رعایت ادب (یک میزبان، یک صف)
self.host_queues = defaultdict(deque)

# زمان آخرین درخواست به هر میزبان
self.last_fetch_time = defaultdict(float)

# آمار کرالر
self.crawled_count = 0
self.total_links_found = 0

def _is_polite(self, host):
    """آیا می‌توان به این میزبان درخواست فرستاد؟"""
    last_time = self.last_fetch_time[host]
    return (time.time() - last_time) >= self.politeness_delay

def _add_to_frontier(self, new_urls, source_url):
    """URL اضافه کردن لینک‌های جدید به مرز"""
    for url in new_urls:
        # حذف تکراری: اگر قبلاً دیدیم یا در مرز است، اضافه نمی‌کنیم. 1.
        if url in self.visited_urls or url in self.url_frontier:
            continue

        # 2. احترام به robots.txt
        if not check_robots_txt(url, self.robots_cache)['allowed']:
            print(f"رد شد robots.txt به دلیل محدودیت {url} لینک ❌")
            continue

        # 3. اضافه به مرز
        self.url_frontier.append(url)
        print(f"به مرز اضافه شد ({source_url} از) {url} لینک جدید ✅")

def crawl_page(self, url):
    """کرال کردن یک صفحه: دریافت، تحلیل و اضافه کردن لینک‌ها"""
    print(f"\n--- [CRAWL] شروع کرال صفحه {url} ---")

    host = urlparse(url).netloc
    # مرحله 1: حل DNS
    ip_address = simulate_dns_lookup(host)

    # مرحله 2: دریافت صفحه
    html_content = simulate_http_fetch(url)

    # مرحله 3: تحلیل صفحه و استخراج لینک‌ها
    new_links = extract_links(html_content, url)

    # مرحله 4: اضافه کردن لینک‌های جدید به مرز
    self._add_to_frontier(new_links, source_url=url)

    # به‌روزرسانی آمار و وضعیت
    self.visited_urls.add(url)
    self.crawled_count += 1
    self.total_links_found += len(new_links)
    self.last_fetch_time[host] = time.time()

    print(f"--- لینک جدید پیدا شد {len(new_links)}، تمام شد {url} کرال صفحه [CRAWL] ---")

def start_crawling(self):
    """شروع فرآیند اصلی کرالینگ"""
    print("...کرالر وب شروع به کار کرد 🚀")

    while self.url_frontier and self.crawled_count < self.max_pages:
        # از مرز URL مرحله 1: انتخاب یک
        current_url = self.url_frontier.popleft()

        host = urlparse(current_url).netloc

        # مرحله 2: رعایت ادب
        if not self._is_polite(host):
            print(f"{host}... منتظر ماندن برای ادب به میزبان 🕒")
            time.sleep(self.politeness_delay)

        # مرحله 3: کرال کردن صفحه
        self.crawl_page(current_url)

    print("\n✅ کرالینگ به پایان رسید.")
    print(f"📊 آمار نهایی:")
    print(f"    {self.crawled_count}: تعداد کل صفحات کرال شده -")
    print(f"    {self.total_links_found}: تعداد کل لینک‌های کشف شده -")
    print(f"    {len(self.url_frontier)}: های باقی‌مانده در مرزURL تعداد -")

# --- مثال اجرا ---
if __name__ == "__main__":
    # های شروعURL لیست (seed set)
    SEED_URLS = [
        "http://example.com/",
        "http://university.edu/departments",
        "http://blog-site.org/index"
```

```
]
```

```
# ساخت و اجرای کرالر
```

```
crawler = WebCrawler(seed_urls=SEED_URLS, max_pages=10, politeness_delay=2)  
crawler.start_crawling()
```

...کراتلر وب شروع به کار کرد 🚀

```
--- [CRAWL] شروع کراتل صفحه http://example.com/ ---
🔍 [DNS] در حال پیدا کردن آدرس example.com...
🌐 [HTTP] http://example.com/... در حال دریافت صفحه از
🤖 [Robots] example.com... برای robots.txt در حال دریافت
✅ به مرز اضافه شد (از http://example.com/) http://example.com/page1.html لینک جدید
🤖 [Robots] example.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://example.com/) http://example.com/page2.html لینک جدید
🤖 [Robots] external-site.com... برای robots.txt در حال دریافت
✅ به مرز اضافه شد (از http://example.com/) http://external-site.com/about لینک جدید
--- [CRAWL] http://example.com/ کراتل صفحه ---

--- [CRAWL] شروع کراتل صفحه http://university.edu/departments ---
🔍 [DNS] در حال پیدا کردن آدرس university.edu...
🌐 [HTTP] http://university.edu/departments... در حال دریافت صفحه از
🤖 [Robots] university.edu... برای robots.txt در حال دریافت
✅ به مرز اضافه شد (از http://university.edu/departments) http://university.edu/page2.html لینک جدید
🤖 [Robots] university.edu برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://university.edu/departments) http://university.edu/page1.html لینک جدید
--- [CRAWL] http://university.edu/departments کراتل صفحه ---

--- [CRAWL] شروع کراتل صفحه http://blog-site.org/index ---
🔍 [DNS] در حال پیدا کردن آدرس blog-site.org...
🌐 [HTTP] http://blog-site.org/index... در حال دریافت صفحه از
🤖 [Robots] blog-site.org... برای robots.txt در حال دریافت
✅ به مرز اضافه شد (از http://blog-site.org/index) http://blog-site.org/page2.html لینک جدید
🤖 [Robots] blog-site.org برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://blog-site.org/index) http://blog-site.org/page1.html لینک جدید
--- [CRAWL] http://blog-site.org/index کراتل صفحه ---

--- [CRAWL] شروع کراتل صفحه http://example.com/page1.html ---
🔍 [DNS] در حال پیدا کردن آدرس example.com...
🌐 [HTTP] http://example.com/page1.html... در حال دریافت صفحه از
🤖 [Robots] example.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://example.com/page1.html) http://example.com/page1.html لینک جدید
--- [CRAWL] http://example.com/page1.html کراتل صفحه ---
⌚ example.com... منتظر ماندن برای ادب به میزبان

--- [CRAWL] شروع کراتل صفحه http://example.com/page2.html ---
🔍 [DNS] در حال پیدا کردن آدرس example.com...
🌐 [HTTP] http://example.com/page2.html... در حال دریافت صفحه از
🤖 [Robots] example.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://example.com/page2.html) http://example.com/page2.html لینک جدید
--- [CRAWL] http://example.com/page2.html کراتل صفحه ---

--- [CRAWL] شروع کراتل صفحه http://external-site.com/about ---
🔍 [DNS] در حال پیدا کردن آدرس external-site.com...
🌐 [HTTP] http://external-site.com/about... در حال دریافت صفحه از
🤖 [Robots] external-site.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://external-site.com/about) http://external-site.com/about لینک جدید
🤖 [Robots] external-site.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://external-site.com/about) http://external-site.com/page2.html لینک جدید
🤖 [Robots] external-site.com برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://external-site.com/about) http://external-site.com/page1.html لینک جدید
--- [CRAWL] http://external-site.com/about کراتل صفحه ---

--- [CRAWL] شروع کراتل صفحه http://university.edu/page2.html ---
🔍 [DNS] در حال پیدا کردن آدرس university.edu...
🌐 [HTTP] http://university.edu/page2.html... در حال دریافت صفحه از
🤖 [Robots] university.edu برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://university.edu/page2.html) http://university.edu/page2.html لینک جدید
--- [CRAWL] http://university.edu/page2.html کراتل صفحه ---
⌚ university.edu... منتظر ماندن برای ادب به میزبان

--- [CRAWL] شروع کراتل صفحه http://university.edu/page1.html ---
🔍 [DNS] در حال پیدا کردن آدرس university.edu...
🌐 [HTTP] http://university.edu/page1.html... در حال دریافت صفحه از
🤖 [Robots] university.edu برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://university.edu/page1.html) http://university.edu/page1.html لینک جدید
--- [CRAWL] http://university.edu/page1.html کراتل صفحه ---

--- [CRAWL] شروع کراتل صفحه http://blog-site.org/page2.html ---
🔍 [DNS] در حال پیدا کردن آدرس blog-site.org...
🌐 [HTTP] http://blog-site.org/page2.html... در حال دریافت صفحه از
🤖 [Robots] blog-site.org برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://blog-site.org/page2.html) http://blog-site.org/page2.html لینک جدید
--- [CRAWL] http://blog-site.org/page2.html کراتل صفحه ---
⌚ blog-site.org... منتظر ماندن برای ادب به میزبان

--- [CRAWL] شروع کراتل صفحه http://blog-site.org/page1.html ---
🔍 [DNS] در حال پیدا کردن آدرس blog-site.org...
🌐 [HTTP] http://blog-site.org/page1.html... در حال دریافت صفحه از
🤖 [Robots] blog-site.org برای robots.txt استفاده از کش
✅ به مرز اضافه شد (از http://blog-site.org/page1.html) http://blog-site.org/page1.html لینک جدید
--- [CRAWL] http://blog-site.org/page1.html کراتل صفحه ---
```

✅ کراتلینگ به پایان رسید.
📊 آمار نهایی:
- تعداد کل صفحات کراتل شده: 10
- تعداد کل لینک‌های کشف شده: 30
- های باقی‌مانده در مرز: URL9 تعداد

In [17]:

```
from requests import Session
from requests.adapters import HTTPAdapter
try:
    # urllib3 v2+
```

```

    from urllib3.util.retry import Retry
except Exception:
    # fallback for some environments
    from requests.packages.urllib3.util.retry import Retry

from bs4 import BeautifulSoup
from collections import deque
from concurrent.futures import ThreadPoolExecutor, wait
from threading import Lock
from typing import List, Optional, Dict, Any
import json
import time
import random

class IMDBcrawler:
    """
    IMDB crawler (Top 250-seeded) that only extracts 4 fields:
    - title : str
    - genres : List[str]
    - stars : List[str]
    - summaries : List[str]

    Networking:
    - Session with retries/backoff for 429/5xx
    - Global rate limiter shared by threads
    """

    BASE_URL = "https://www.imdb.com"
    TOP_250_URL = f"{BASE_URL}/chart/top/"

    headers = {
        "User-Agent": (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
            "AppleWebKit/537.36 (KHTML, like Gecko) "
            "Chrome/122.0.0.0 Safari/537.36"
        )
    }

    def __init__(
        self,
        crawling_threshold: int = 1000,
        *,
        max_workers: int = 3,
        min_interval: float = 1.0,
        timeout: int = 30,
        retries: int = 6
    ) -> None:
        self.crawling_threshold = crawling_threshold
        self.not_crawled: deque[str] = deque()
        self.crawled: List[Dict[str, Any]] = []
        self.added_ids: set[str] = set()

        self.add_list_lock = Lock()
        self.add_queue_lock = Lock()

        self.max_workers = max_workers
        self.min_interval = float(min_interval)
        self.timeout = timeout
        self.session = self._build_session(retries)

        self._last_request_ts = 0.0
        self._rate_lock = Lock()

    # ----- Networking -----

    def _build_session(self, retries: int) -> Session:
        s = Session()
        try:
            retry = Retry(
                total=retries,
                connect=retries,
                read=retries,
                status=retries,
                backoff_factor=1.0,
                status_forcelist=[429, 500, 502, 503, 504],
                allowed_methods=frozenset({"GET"}),
                raise_on_status=False,
            )
        except TypeError:
            retry = Retry(
                total=retries,
                connect=retries,
                read=retries,
                status=retries,
                backoff_factor=1.0,
                status_forcelist=[429, 500, 502, 503, 504],
                method_whitelist=frozenset({"GET"}),
                raise_on_status=False,
            )
        adapter = HTTPAdapter(max_retries=retry, pool_connections=100, pool_maxsize=100)
        s.mount("https://", adapter)
        s.mount("http://", adapter)
        s.headers.update({
            "User-Agent": self.headers.get("User-Agent", ""),
            "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
            "Accept-Language": "en-US,en;q=0.9",

```

```

        "Accept-Encoding": "gzip, deflate, br",
        "Connection": "keep-alive",
        "Referer": "https://www.imdb.com/",
    })
    return s

def crawl(self, url: str):
    """GET with retries/timeout + GLOBAL rate limit."""
    with self._rate_lock:
        now = time.time()
        wait_time = self.min_interval - (now - self._last_request_ts)
        if wait_time > 0:
            time.sleep(wait_time)
        time.sleep(random.uniform(0.05, 0.25)) # jitter
        self._last_request_ts = time.time()

    try:
        resp = self.session.get(url, timeout=self.timeout)
        if resp.status_code == 200:
            return resp
        print(f"Failed to crawl {url}: Status code {resp.status_code}")
    except Exception as e:
        print(f"An error occurred while crawling {url}: {e}")
    return None

# ----- Seeds -----

def extract_top_250(self) -> None:
    """Seed the queue from IMDb Top 250 page."""
    try:
        response = self.crawl(self.TOP_250_URL)
        if response is None:
            print("Failed to retrieve Top 250 movies list.")
            return

        soup = BeautifulSoup(response.content, "html.parser")
        movies = soup.find_all("li", class_="ipc-metadata-list-summary-item")

        self.not_crawled.clear()
        self.added_ids.clear()

        for movie in movies:
            link = movie.find("a", href=True)
            if not link:
                continue
            href = link.get("href", "")
            parts = href.split("/")
            if len(parts) > 2:
                movie_id = parts[2]
                if movie_id and movie_id not in self.added_ids:
                    self.not_crawled.append(f"{self.BASE_URL}/title/{movie_id}/")
                    self.added_ids.add(movie_id)
    except Exception as e:
        print(f"An error occurred while extracting Top 250 movies: {e}")

# ----- Public -----

def start_crawling(self) -> None:
    """Crawl titles from Top 250 (and related) until threshold/queue empty."""
    self.extract_top_250()

    futures = []
    crawled_counter = 0

    with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
        while crawled_counter < self.crawling_threshold and self.not_crawled:
            url = self.not_crawled.popleft()
            futures.append(executor.submit(self.crawl_page_info, url, crawled_counter))
            crawled_counter += 1

            if not self.not_crawled:
                wait(futures)
                futures.clear()

        if futures:
            wait(futures)

def crawl_page_info(self, url: str, crawler_counter: int) -> Optional[Dict[str, Any]]:
    """
    Crawl a single title page and extract ONLY:
    - title
    - genres
    - stars
    - summaries
    Related links may be appended to the queue.
    """
    response = self.crawl(url)
    if response is None:
        print(f"Failed to crawl page: {url}")
        return None

    soup = BeautifulSoup(response.content, "html.parser")

    movie = self.get_imdb_instance()
    movie["id"] = self.get_id_from_URL(url)
    movie["title"] = self.get_title(soup)
    movie["genres"] = self.get_genres(soup)

```

```

movie["stars"] = self.get_stars(soup)

# Prefer full plot summaries page; fallback to first page short plot if empty
summaries = self.get_summaries(url)
if not summaries:
    short = self.get_first_page_summary(soup)
    if short and short != "Summary not found.":
        summaries = [short]
movie["summaries"] = summaries

with self.add_list_lock:
    self.crawled.append(movie)

# Optional queue expansion via "More like this"
related_links = self.get_related_links(soup)
if related_links:
    with self.add_queue_lock:
        for link in related_links:
            movie_id = self.get_id_from_URL(link)
            if movie_id and movie_id not in self.added_ids:
                self.not_crawled.append(link)
                self.added_ids.add(movie_id)

print(crawler_counter, f"Successfully crawled and processed: {url}")
return movie

# ----- Serialization -----

def save_to_json(self) -> None:
    """Write crawled results to a JSON file."""
    try:
        with open("IMDB_crawled.json", "w", encoding="utf-8") as f:
            json.dump(self.crawled, f, ensure_ascii=False, indent=4)
        print(f"✅ ذخیره شد IMDB_crawled.json فیلم با موفقیت در فایل {len(self.crawled)} اطلاعات")
    except Exception as e:
        print(f"An error occurred while writing to JSON file: {e}")

# ----- Extraction helpers -----

@staticmethod
def get_imdb_instance() -> Dict[str, Any]:
    return {
        "id": None,
        "title": None,
        "genres": None,
        "stars": None,
        "summaries": None,
    }

@staticmethod
def get_id_from_URL(url: str) -> Optional[str]:
    """https://www.imdb.com/title/tt0111161/ -> tt0111161"""
    try:
        parts = url.split("/")
        title_index = parts.index("title")
        if title_index + 1 < len(parts):
            return parts[title_index + 1]
        print("Error: 'title' segment found, but no ID follows in the URL.")
    except ValueError:
        print("Error: The URL does not contain a 'title' segment.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
    return None

def get_summary_link(self, url: str) -> Optional[str]:
    """ /title/<id>/ -> /title/<id>/plotsummary """
    try:
        movie_id = self.get_id_from_URL(url)
        if movie_id:
            return f"{self.BASE_URL}/title/{movie_id}/plotsummary"
        print("Failed to extract movie ID from URL.")
    except Exception as e:
        print(f"Failed to get summary link: {e}")
    return None

def get_related_links(self, soup: BeautifulSoup) -> List[str]:
    """Collect 'More like this' title links to expand crawling (optional)."""
    try:
        related_links: List[str] = []
        section = soup.find("section", {"data-testid": "MoreLikeThis"})
        if not section:
            return related_links
        links = section.find_all("a", class_="ipc-lockup-overlay ipc-focusable")
        for link in links:
            href = link.get("href")
            if href:
                related_links.append(f"https://www.imdb.com{href}")
        return related_links
    except Exception as e:
        print(f"Failed to get related links: {e}")
    return []

@staticmethod
def get_title(soup: BeautifulSoup) -> Optional[str]:
    try:
        tag = soup.find("h1")
        return tag.text.strip() if tag else None
    
```

```

        except Exception as e:
            print(f"Failed to get title: {e}")
            return None

    @staticmethod
    def get_first_page_summary(soup: BeautifulSoup) -> str:
        try:
            tag = soup.find("span", {"data-testid": "plot-xl"})
            return tag.text.strip() if tag else "Summary not found."
        except Exception as e:
            print(f"Failed to get summary: {e}")
            return "Summary not found."

    @staticmethod
    def _get_principal_names(soup: BeautifulSoup, label_text: str) -> List[str]:
        """Extract names (e.g., Stars) from principal credit blocks."""
        try:
            names: List[str] = []
            blocks = soup.find_all("li", {"data-testid": "title-pc-principal-credit"})
            for block in blocks:
                for tag_name, cls in [
                    ("span", "ipc-metadata-list-item__label"),
                    ("a", "ipc-metadata-list-item__label"),
                ]:
                    label_tag = block.find(tag_name, class_=cls)
                    if label_tag and label_tag.text.strip() == label_text:
                        for a in block.find_all("a", class_="ipc-metadata-list-item__list-content-item"):
                            txt = a.text.strip()
                            if txt:
                                names.append(txt)
            return list(set(names)) if names else []
        except Exception as e:
            print(f"Failed to get {label_text.lower()}: {e}")
            return []

    def get_stars(self, soup: BeautifulSoup) -> List[str]:
        return self._get_principal_names(soup, "Stars")

    @staticmethod
    def get_genres(soup: BeautifulSoup) -> List[str]:
        genres: List[str] = []
        try:
            scroller = soup.find("div", class_="ipc-chip-list__scroller")
            if not scroller:
                return genres
            for span in scroller.find_all("span", class_="ipc-chip__text"):
                txt = span.text.strip()
                if txt:
                    genres.append(txt)
        except Exception as e:
            print(f"Failed to get genres: {e}")
        return genres

    def get_summaries(self, url: str) -> List[str]:
        summaries: List[str] = []
        try:
            summaries_url = self.get_summary_link(url)
            if not summaries_url:
                return summaries
            resp = self.crawl(summaries_url)
            if not resp:
                return summaries
            soup = BeautifulSoup(resp.content, "html.parser")
            section = soup.find("div", {"data-testid": "sub-section-summaries"})
            if not section:
                return summaries
            for li in section.find_all("li"):
                div = li.find("div", class_="ipc-html-content-inner-div")
                if div and div.text:
                    summaries.append(div.text.strip())
        except Exception as e:
            print(f"Failed to get summaries: {e}")
        return summaries

def main() -> None:
    imdb_crawler = IMDBCrawler(
        crawling_threshold=10, # فقط ۱۰ فیلم صرفاً جهت تمرین
        max_workers=3,
        min_interval=1.0,
        timeout=30,
        retries=6,
    )
    imdb_crawler.start_crawling()

    ذخیره اطلاعات در فایل پس از اتمام کراپینگ
    imdb_crawler.save_to_json()

if __name__ == "__main__":
    main()

```

0 Successfully crawled and processed: https://www.imdb.com/title/tt0111161/
1 Successfully crawled and processed: https://www.imdb.com/title/tt0068646/
2 Successfully crawled and processed: https://www.imdb.com/title/tt0468569/
3 Successfully crawled and processed: https://www.imdb.com/title/tt0071562/
4 Successfully crawled and processed: https://www.imdb.com/title/tt0050083/
5 Successfully crawled and processed: https://www.imdb.com/title/tt0167260/
6 Successfully crawled and processed: https://www.imdb.com/title/tt0108052/
8 Successfully crawled and processed: https://www.imdb.com/title/tt0110912/
7 Successfully crawled and processed: https://www.imdb.com/title/tt0120737/
9 Successfully crawled and processed: https://www.imdb.com/title/tt0060196/
✓ ذخیره شد IMDB_crawled.json 10 فیلم با موفقیت در فایل

20.3 توزیع ایندکس‌ها: معماری پشت موتورهای جستجوی بزرگ

چالش گوگل: مدیریت میلیاردها صفحه در هزاران سرور

تصور کنید مهندسی در گوگل هستید و باید سیستمی طراحی کنید که بتواند به میلیاردها صفحه وب پاسخ دهد. هر روز حدود ۳.۵ میلیارد جستجو در گوگل انجام می‌شود و هر جستجو باید در کسری از ثانیه پاسخ داده شود. برای حل این چالش، گوگل و سایر شرکت‌های بزرگ جستجو از **توزیع ایندکس‌ها** در خوشه‌های بزرگ کامپیوتری استفاده می‌کنند. دو استراتژی اصلی برای این کار وجود دارد:

تقسیم‌بندی بر اساس واژه‌ها

در این روش، واژگان ایندکس به زیرمجموعه‌هایی تقسیم می‌شوند و هر سرور مسئول واژه‌های خاصی است. برای مثال، یک سرور ممکن است واژه‌هایی که با "آ" شروع می‌شوند را نگهداری کند، در حالی که سرور دیگر مسئول واژه‌هایی با "ب" است.

در عمل، این روش با چالش‌های جدی روبرو است:

- وقتی کاربر عبارت "خرید لپ‌تاپ" را جستجو می‌کند، سیستم باید لیست‌های بزرگ را بین سرورهای مختلف منتقل کند.
- توزیع بار به صورت یکنواخت دشوار است، زیرا برخی واژه‌ها (مانند "news") بسیار محبوب‌تر از سایر واژه‌ها هستند.

تقسیم‌بندی بر اساس اسناد

در این روش که در شرکت‌هایی مانند گوگل و مایکروسافت رایج‌تر است، هر سرور ایندکس کامل برای زیرمجموعه‌ای از اسناد را نگهداری می‌کند. برای مثال، سرور شماره ۱ ممکن است مسئول ایندکس تمام صفحات وب‌سایت‌های خبری باشد، در حالی که سرور شماره ۲ مسئول وبلاگ‌های شخصی است. وقتی کاربر عبارتی را جستجو می‌کند، پرس‌وجو به همه سرورها ارسال می‌شود و نتایج از هر سرور جمع‌آوری و ادغام می‌شوند.

چگونه اسناد را بین سرورها توزیع کنیم؟

در سیستم تقسیم‌بندی بر اساس اسناد، دو روش اصلی برای توزیع اسناد بین سرورها وجود دارد:

- توزیع بر اساس میزبان:** تمام صفحات یک دامنه (مانند wikipedia.org) به یک سرور اختصاص می‌یابند. این روش ساده است، اما مشکلی جدی دارد: بسیاری از جستجوها نتایجشان از تعداد کمی دامنه‌های محبوب (مانند آمازون، یوتیوب و فیسبوک) می‌آید، بنابراین بار محاسباتی به صورت یکنواخت توزیع نمی‌شود.

- **توزیع بر اساس هش URL:** هر URL با استفاده از یک تابع هش به یک سرور نگاشت می‌شود. این روش تضمین می‌کند که بار محاسباتی به صورت یکنواخت بین سرورها توزیع شود. برای مثال، URLهای وبسایت‌های مختلف به صورت تصادفی بین سرورها پخش می‌شوند.

🏆 تکنیک پیشرفته: تقسیم‌بندی سلسله‌مراتبی

برای بهینه‌سازی بیشتر، شرکت‌هایی مانند گوگل از تکنیک **تقسیم‌بندی سلسله‌مراتبی** استفاده می‌کنند:

1. اسناد بر اساس احتمال رتبه‌بندی بالا به دو دسته تقسیم می‌شوند: اسناد باکیفیت تر و اسناد باکیفیت.
2. اسناد باکیفیت تر در **ایندکس‌های اولیه** قرار می‌گیرند که معمولاً در حافظه‌های سریع‌تر (مانند SSD) ذخیره می‌شوند.
3. اسناد باکیفیت در **ایندکس‌های ثانویه** ذخیره می‌شوند که ممکن است در حافظه‌های کندتر (مانند HDD) قرار داشته باشند.
4. هنگام جستجو، ابتدا فقط ایندکس‌های اولیه جستجو می‌شوند. اگر نتایج کافی نبود، سیستم به جستجو در ایندکس‌های ثانویه می‌پردازد.

این روش مانند جستجو در یک کتابخانه بزرگ است که ابتدا در بخش مراجع معتبر جستجو می‌کنید و فقط در صورت نیاز، به بخش‌های کمتر مرجع مراجعه می‌کنید.

📊 چالش آمار سراسری در سیستم توزیع‌شده

یکی از چالش‌های مهم در سیستم توزیع‌شده، محاسبه آمارهای سراسری مانند **idf** (Inverse Document Frequency) است که برای رتبه‌بندی نتایج جستجو ضروری است.

برای حل این مشکل، شرکت‌ها از فرآیندهای پس‌زمینه‌ای استفاده می‌کنند که به صورت دوره‌ای آمارهای سراسری را محاسبه کرده و به سرورهای مختلف ارسال می‌کنند. این فرآیندها معمولاً در ساعات کم‌ترافیک شبکه اجرا می‌شوند تا تأثیری بر عملکرد سیستم نداشته باشند.

20.4 سرورهای همبستگی: نقشه راهنمای وب 🗺️

📖 چرا گوگل به نقشه وب نیاز دارد؟

تصور کنید در تیم مهندسی گوگل کار می‌کنید و می‌خواهید الگوریتم PageRank را بهبود دهید. برای این کار، باید بدانید کدام وبسایت‌های معتبر به صفحه اصلی دانشگاه استنفورد لینک داده‌اند. یا شاید بخواهید بدانید کدام صفحات از سایت bbc.com به یک خبر خاص اشاره دارند.

این پرس‌وجوها که به آنها **پرس‌وجوهای همبستگی** (Connectivity Queries) می‌گویند، برای موتورهای جستجو حیاتی هستند. اما پاسخ به آنها چالش‌برانگیز است، زیرا نیاز به ذخیره‌سازی کل گراف وب (که میلیاردها صفحه و

📁 چالش ذخیره‌سازی: وقتی وب بزرگتر از حافظه است

بیاپید ابعاد این چالش را درک کنیم. اگر وب شامل ۴ میلیارد صفحه باشد و هر صفحه به طور متوسط ۱۰ لینک داشته باشد، برای ذخیره‌سازی هر لینک (با شناسه مبدأ و مقصد ۳۲ بیتی)، به ۳۲۰ گیگابایت حافظه نیاز داریم! این مانند تلاش برای بارگذاری کل کتابخانه کنگره در RAM یک لپ‌تاپ است.

راه‌حل، فشرده‌سازی هوشمندانه‌ای است که هم فضا را کاهش دهد و هم پرس‌وجوهای سریع را پشتیبانی کند. این تفاوت اصلی با فشرده‌سازی معمولی است؛ ما نه تنها داده‌ها را فشرده می‌کنیم، بلکه باید بتوانیم به سرعت به آنها دسترسی پیدا کنیم.

🔗 سه تکنیک کلیدی برای فشرده‌سازی گراف وب

مهندسان شرکت‌هایی مانند گوگل و آمازون از ترکیبی از سه تکنیک هوشمندانه برای حل این مشکل استفاده می‌کنند:

1. فریب هوشمندانه: کدگذاری فاصله‌ها (Gap Encoding)

به جای ذخیره شماره مطلق هر صفحه مقصد، تفاضل آن با صفحه قبلی را ذخیره می‌کنیم. چون لیست‌ها مرتب هستند، این فاصله‌ها (گپ‌ها) معمولاً اعداد کوچکی هستند و فضای بسیار کمتری اشغال می‌کنند. این تکنیکی است که در فشرده‌سازی ایندکس‌های معکوس نیز استفاده می‌شود.

2. پیدا کردن الگوها: شباهت بین ردیف‌ها

بسیاری از وب‌سایت‌ها از الگوی یکسانی برای ساختار خود استفاده می‌کنند. برای مثال، تقریباً تمام صفحات دانشگاه استنفورد لینک‌هایی به صفحه اصلی، نقشه سایت، و صفحه کپی‌رایت دارند. به جای ذخیره این لیست برای هر صفحه، می‌توانیم یک ردیف نمونه (prototype) ایجاد کرده و تفاوت‌های هر صفحه را با آن نمونه ذخیره کنیم.

3. هم‌جواری: لینک‌های محلی

بسیاری از لینک‌ها به صفحات هم‌میزبان اشاره می‌کنند. در لیست مرتب‌شده URLها، این یعنی شماره‌های مقصد نزدیک به هم هستند و گپ‌های کوچکتری تولید می‌شود، که فشرده‌سازی را مؤثرتر می‌کند.

🏗 معماری عملی: از تئوری تا پیاده‌سازی

این تکنیک‌ها چگونه در عمل ترکیب می‌شوند؟

1. مرتب‌سازی و شماره‌گذاری: ابتدا تمام URLها را به صورت لکسیکوگرافیک مرتب کرده و به هر کدام یک شماره منحصر به فرد (ID عددی) می‌دهیم. این کار پایه و اساس سیستم ماست.
2. ایجاد جدول مجاورت: جدولی ایجاد می‌کنیم که هر ردیف آن شامل لیست مرتب‌شده‌ای از ID صفحاتی است که به آن صفحه لینک داده‌اند (in-links) یا از آن لینک گرفته‌اند (out-links).

3. **کدگذاری هوشمند:** از بالا به پایین جدول را پیمایش می‌کنیم و هر ردیف را بر اساس ۷ ردیف قبلی

فشرده می‌کنیم. چرا ۷؟ چون صفحات یک وب‌سایت در این مرتب‌سازی کنار هم قرار می‌گیرند و ۷

ردیف قبلی به احتمال زیاد به همان سایت تعلق دارند. این محدودیت باعث می‌شود جستجو برای

ردیف نمونه سریع باشد (فقط ۳ بیت برای ذخیره Offset نیاز است).

4. **بازسازی در زمان پرس‌وجو:** هنگام پاسخ به یک پرس‌وجو، ابتدا ID صفحه مورد نظر را پیدا کرده، سپس

ردیف فشرده‌شده آن را بازسازی می‌کنیم. اگر ردیف به یک ردیف نمونه اشاره کند، آن ردیف نمونه نیز

بازسازی می‌شود.

این تکنیک‌ها به‌طور ترکیبی، مصرف حافظه را از ۶۴ بیت به ازای هر لینک در روش ساده، به تنها ۳ بیت به ازای

هر لینک کاهش می‌دهند - یک پیشرفت چشمگیر!

⚖️ تعادل بین فشرده‌سازی و سرعت

یک چالش مهم در این سیستم، پیدا کردن تعادل مناسب است. اگر آستانه شباهت برای انتخاب ردیف نمونه

خیلی بالا باشد، از این تکنیک به ندرت استفاده می‌شود و فشرده‌سازی ضعیف می‌شود. اگر آستانه خیلی پایین

باشد، ردیف‌های زیادی به ردیف‌های نمونه اشاره می‌کنند و بازسازی یک ردیف ساده ممکن است به چندین سطح

از ارجاعات نیاز داشته باشد که سرعت را کاهش می‌دهد.

این تعادل دقیق یکی از رازهای موفقیت موتورهای جستجوی مدرن است که به آنها اجازه می‌دهد کل وب را در

حافظه مدیریت کرده و به پرس‌وجوهای پیچیده در کسری از ثانیه پاسخ دهند.

In [2]: شبیه‌سازی فشرده‌سازی گراف وب #

```
import random
from typing import List, Tuple, Optional, Dict

class WebGraphCompressor:
    """
    شبیه‌سازی فشرده‌سازی گراف وب با سه تکنیک:
    1. کدگذاری گپ (Gap Encoding)
    2. کدگذاری نسبت به ردیف نمونه (Prototype-based Encoding)
    """

    def __init__(self, similarity_threshold=0.6):
        self.similarity_threshold = similarity_threshold
        self.compressed_rows = [] # لیست ردیف‌های فشرده‌شده
        self.url_to_id = {} # عددی ID به URL نگاشت
        self.id_to_url = [] # URL به ID نگاشت
        self.next_id = 0

    def register_url(self, url: str) -> int:
        """عدد منحصربه‌فرد آن ID و بازگرداندن URL ثبت یک"""
        if url not in self.url_to_id:
            self.url_to_id[url] = self.next_id
            self.id_to_url.append(url)
            self.next_id += 1
        return self.url_to_id[url]

    def encode_gap(self, sorted_ids: List[int]) -> List[int]:
        """کدگذاری گپ: تبدیل لیست شماره‌های مرتب به تفاضل‌های متوالی"""
        if not sorted_ids:
            return []
        gaps = [sorted_ids[0]]
        for i in range(1, len(sorted_ids)):
            gaps.append(sorted_ids[i] - sorted_ids[i-1])
        return gaps

    def decode_gap(self, gaps: List[int]) -> List[int]:
        """بازگردانی لیست اصلی از روی گپ‌ها"""
        if not gaps:
            return []
        ids = [gaps[0]]
        for i in range(1, len(gaps)):
            ids.append(ids[-1] + gaps[i])
```

```

        return ids

def calculate_similarity(self, row1: List[int], row2: List[int]) -> float:
    """محاسبه شباهت بین دو ردیف بر اساس اشتراک مجموعه‌ای"""
    if not row1 or not row2:
        return 1.0
    set1, set2 = set(row1), set(row2)
    intersection = len(set1 & set2)
    union = len(set1 | set2)
    return intersection / union if union > 0 else 0.0

def find_best_prototype(self, current_row: List[int], window_size: int = 7) -> Optional[Tuple[int, Dict[str, List[int]]]]:
    """
    ردیف آخر 'window_size' یافتن بهترین ردیف نمونه از بین
    . اگر شباهت کافی وجود نداشته باشد None یا (offset, differences) بازگشت
    """
    if not self.compressed_rows:
        return None

    num_rows = len(self.compressed_rows)
    start_idx = max(0, num_rows - window_size)

    best_similarity = 0.0
    best_offset = 0
    best_diff = {"remove": [], "add": []}

    for i in range(num_rows - 1, start_idx - 1, -1):
        # بازسازی ردیف نمونه (همیشه ذخیره‌شده به صورت گپ)
        prototype_gaps = self.compressed_rows[i]["data"]
        prototype_row = self.decode_gap(prototype_gaps)

        similarity = self.calculate_similarity(current_row, prototype_row)

        if similarity > best_similarity:
            best_similarity = similarity
            best_offset = num_rows - i # فاصله از ردیف فعلی

            # محاسبه تفاوت‌ها: عناصری که باید حذف/افزوده شوند
            remove = list(set(prototype_row) - set(current_row))
            add = list(set(current_row) - set(prototype_row))
            best_diff = {"remove": sorted(remove), "add": sorted(add)}

    if best_similarity >= self.similarity_threshold:
        return (best_offset, best_diff)
    return None

def add_adjacency_row(self, out_links: List[str]) -> None:
    """
    افزودن یک ردیف مجاورت (لیست لینک‌های خروجی یک صفحه) به گراف فشرده‌شده
    """
    # های عددی ID ها به URL تبدیل
    link_ids = [self.register_url(url) for url in out_links]
    link_ids.sort() # لیست باید مرتب باشد

    # تلاش برای یافتن یک ردیف نمونه مناسب
    prototype_info = self.find_best_prototype(link_ids)

    # اگر ردیف نمونه مناسبی پیدا شد، آن را فشرده می‌کنیم
    if prototype_info is not None:
        offset, diff = prototype_info
        self.compressed_rows.append({
            "type": "prototype",
            "offset": offset,
            "diff": diff
        })
    else:
        # اگر ردیف نمونه مناسبی نبود، از کدگذاری گپ استفاده می‌کنیم
        gaps = self.encode_gap(link_ids)
        self.compressed_rows.append({
            "type": "gap",
            "data": gaps
        })

def reconstruct_row(self, row_index: int) -> List[int]:
    """
    بازسازی لیست لینک‌های یک ردیف از فرم فشرده‌شده
    """
    if row_index >= len(self.compressed_rows) or row_index < 0:
        return []

    row = self.compressed_rows[row_index]
    if row["type"] == "gap":
        return self.decode_gap(row["data"])
    elif row["type"] == "prototype":
        # بازسازی ردیف نمونه اول
        prototype_index = row_index - row["offset"]
        prototype_row = self.reconstruct_row(prototype_index)

        # اعمال تغییرات
        result_set = set(prototype_row)
        for item in row["diff"]["remove"]:
            result_set.discard(item)
        for item in row["diff"]["add"]:
            result_set.add(item)
        return sorted(list(result_set))
    return []

```

```
def get_original_links(self, row_index: int) -> List[str]:
    """برای یک ردیف URL بازگرداندن لینک‌های اصلی به صورت"""
    ids = self.reconstruct_row(row_index)
    return [self.id_to_url[i] for i in ids if i < len(self.id_to_url)]

# --- شبیه‌سازی سرور همبستگی ---
class ConnectivityServer:
    """
    شبیه‌سازی یک سرور همبستگی که پرس‌وجوهای را مدیریت می‌کند
    """

    def __init__(self, compressor: WebGraphCompressor):
        self.compressor = compressor
        self.inverted_index = {} # نگاشت لینک -> لیست اسناد

    def add_document(self, doc_id: int, links: List[str]):
        """افزودن یک سند جدید به ایندکس معکوس"""
        for link in links:
            # --- اصلاح: استفاده از متدهای دیکشنری ---
            if link not in self.inverted_index:
                self.inverted_index[link] = set()
            self.inverted_index[link].add(doc_id)

    def handle_query(self, query_links: List[str]) -> List[int]:
        """
        پاسخ به یک پرس‌وجو: لیست اسندهایی که شامل تمام لینک‌های پرس‌وجو هستند
        """
        result_sets = [set(self.inverted_index.get(link, set())) for link in query_links]

        if not result_sets:
            return []

        # پیدا کردن اشتراک بین تمام مجموعه‌ها
        common_docs = result_sets[0]
        for i in range(1, len(result_sets)):
            common_docs.intersection_update(result_sets[i])

        # تبدیل مجموعه اسناد به لیست مرتب
        return sorted(list(common_docs))

# --- سناریوی اصلی ---
def main():
    """
    سناریوی اصلی: ایجاد گراف، فشردسازی، و پاسخ به پرس‌وجوها
    """

    # ۱. تعریف صفحات و لینک‌ها (مطابق شکل ۲۰.۵)
    pages = {
        1: {"id": 1, "links": [2, 3, 4, 8, 16, 32, 64]},
        2: {"id": 2, "links": [1, 3, 5, 6, 7, 8, 9]},
        3: {"id": 3, "links": [1, 4, 5, 6, 7, 8]},
        4: {"id": 4, "links": [1, 2, 3, 4, 5, 6]},
        5: {"id": 5, "links": [1, 2, 3, 4, 5]},
        6: {"id": 6, "links": [1, 2, 3, 4]}
    }

    # ۲. ایجاد فشردساز و سرور
    compressor = WebGraphCompressor()
    server = ConnectivityServer(compressor)

    # ۳. افزودن اسناد به سرور
    for doc_id, page_data in pages.items():
        server.add_document(doc_id, page_data["links"])

    # ۴. سناریوی اصلی: پاسخ به پرس‌وجوها
    queries = [
        [2, 4], # پرس‌وجو: صفحاتی که به صفحه ۲ لینک دارند
        [1, 5, 6], # پرس‌وجو: صفحاتی که به صفحات ۱، ۳ و ۵ لینک دارند
        [3, 4, 5], # پرس‌وجو: صفحاتی که به صفحات ۲، ۳ و ۴ لینک دارند
    ]

    print("شبیه‌سازی سرور همبستگی 🔄")
    print("-" * 40)

    for i, query in enumerate(queries):
        print(f"\nپرس‌وجو {i+1}: {query} -> {server.handle_query(query)}")
        result = server.handle_query(query)
        print(f"پاسخ: {result}")

if __name__ == "__main__":
    main()

print("بسیار ناکارآمد است و در مقیاس‌های بزرگ کارایی ندارد (Query Processing) نکته مهم: تکنیک فشردسازی گراف، هرچقدر قدرتمند، برای کاهش حجم داده‌ها عالی است، اما در عملکرد جستجو
```

شبیه‌سازی سرور همبستگی: -----

پرس‌وجو 1: [4, 2] -> [6, 5, 4, 1]
پاسخ: [6, 5, 4, 1]

پرس‌وجو 2: [6, 5, 1] -> [4, 3, 2]
پاسخ: [4, 3, 2]

پرس‌وجو 3: [5, 4, 3] -> [5, 4]
پاسخ: [5, 4]

بسیار ناکارآمد است و در مقیاس‌های بزرگ کارایی ندارد (Query Processing) نکته مهم: تکنیک فشردسازی گراف، هرچقدر قدرتمند، برای کاهش حجم داده‌ها عالی است، اما در عملکرد جستجو

جمع‌بندی فصل ۲۰: خزنده‌ها و ایندکس‌های وب ✦

در این فصل:

- فرآیند **خزنده‌برداری وب** یا **به اختصار خزش وب** (Web crawling) را به‌عنوان گامی حیاتی برای جمع‌آوری صفحات وب و ساخت ایندکس معرفی کردیم.
- ویژگی‌های ضروری خزنده‌ها را بررسی کردیم: – **ربات‌گیر** (Robustness): مقاومت در برابر دام‌های خزنده (spider traps)، – **ادب** (Politeness): رعایت محدودیت‌های سرورها در سرعت درخواست.
- معماری یک خزنده واقعی را تحلیل کردیم: – **مرز** (URL frontier) برای مدیریت صف درخواست‌ها، – مازول‌های **DNS**، **دریافت** (fetch)، **تجزیه** (parse) و **حذف تکرار** (duplicate elimination).
- چالش‌های خزنده‌های توزیع‌شده را پوشش دادیم: – پارتیشن‌بندی میزبان‌ها بین گره‌ها، – مدیریت سراسری مجموعه اثرانگشت‌ها برای شناسایی صفحات تکراری.
- راهکارهای هوشمند برای **مرز URL** را معرفی کردیم: – صف‌های جلو برای **اولویت‌بندی** (کیفیت و تازگی)، – صف‌های عقب برای رعایت **ادب** (فاصله زمانی بین درخواست‌ها به یک میزبان).
- دو روش اصلی **توزیع ایندکس** را مقایسه کردیم: – پارتیشن‌بندی بر اساس **اصطلاحات** (term partitioning) – چالش‌برانگیز برای پرسمان‌های چندکلمه‌ای، – پارتیشن‌بندی بر اساس **اسناد** (document partitioning) – رایج‌تر و ساده‌تر برای پیاده‌سازی.
- نیاز به **سرورهای اتصال** (Connectivity servers) را تبیین کردیم: ذخیره ساختار گراف وب برای پشتیبانی از تحلیل پیوندها (فصل ۲۱) با استفاده از فشرده‌سازی هوشمند لیست‌های مجاورت.

این فصل پایه‌های فنی ساخت یک موتور جست‌وجوی وب را فراهم می‌کند. در فصل بعدی، با مفاهیمی مانند:

- تحلیل ساختار گراف وب برای رتبه‌بندی (PageRank و HITS)
- الگوریتم‌های تشخیص و مبارزه با اسپم پیوندی (Link spam)
- استفاده از متن لنگر (Anchor text) به‌عنوان سیگنال رتبه‌بندی

آشنا خواهیم شد – که همگی بر همین پایه داده‌های جمع‌آوری‌شده توسط خزنده‌ها و ایندکس‌های توزیع‌شده بنا شده‌اند.